

MedPredict

AI Disease Risk Prediction System

A Technical Project Report submitted in partial fulfilment of the
requirements for the award of the degree of

Bachelor of Technology
Computer Science & Engineering

Submitted by

Student Name	Registration Number
Pawan Kumar	22105125032
Aryan Ankit	22105125008
Naveen Prakash	22105125011
Shekhar Sanu	22105125013
Shatrudhan Kumar	22105125009

Under the Guidance of

Annu Kumari
Assistant Professor, Department of Computer Science & Engineering

Department of Computer Science & Engineering
[Rashatrakavi Ramdhari Singh Dinkar College of Engineering]

Batch: 2022-2026

Certificate

This is to certify that the project entitled "MedPredict — AI Disease Risk Prediction System" submitted by the following group of students to the Department of Computer Science and Engineering, [Rashatrakavi Ramdhari Singh Dinkar College of Engineering], in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by them under my supervision and guidance during the academic year 2025–2026.

Student Name	Enrollment Number	Primary Contribution
Pawan Kumar	22105125032	Backend Development & Model Training
Aryan Ankit	22105125008	Frontend Design & UI/UX
Naveen Prakash	22105125011	Data Preprocessing & EDA
Shekhar Sanu	22105125013	Model Evaluation & Testing
Shatrudhan Kumar	22105125009	Deployment & Documentation

The results embodied in this project report have not been submitted to any other university or institution for the award of any degree or diploma. This work is original and has been carried out under the direct supervision of the undersigned supervisor.

Signature of Supervisor: _____

Name: Annu Kumari

Designation: Assistant Professor

Department of Computer Science & Engineering

Date: _____

Signature of Head of Department: _____

Name: Prof. Md Shahid Ahmad

Date: _____

Declaration

We, the undersigned students of Bachelor of Technology in Computer Science and Engineering at [Rashatrakavi Ramdhari Singh Dinkar College of Engineering], hereby declare that the project work entitled "MedPredict — AI Disease Risk Prediction System" submitted in partial fulfilment of the requirements for the award of the degree of B.Tech. (CSE) is an authentic record of our own work carried out during the academic year 2025–2026 under the supervision of Annu Kumari, Assistant Professor, Department of Computer Science and Engineering.

We further declare that the project or any part thereof has not been submitted for any degree, diploma, or similar award to any university or institution in India or abroad. All references, sources of information, and datasets used in this report have been duly acknowledged. We take full responsibility for the originality of the content presented herein.

Date: _____

Place: _____

Student Name	Registration No.
Pawan Kumar	22105125032
Aryan Ankit	22105125008
Naveen Prakash	22105125011
Shekhar Sanu	22105125013
Shatrudhan Kumar	22105125009

Acknowledgement

We express our sincere and deepest gratitude to our project supervisor, Annu Kumari, Assistant Professor, Department of Computer Science and Engineering, [Rashatrakavi Ramdhari Singh Dinkar College of Engineering], for her invaluable guidance, consistent encouragement, timely feedback, and constructive suggestions throughout the development of this project. Her expertise in machine learning, healthcare informatics, and software engineering has been instrumental in shaping both the technical direction and the quality of this work. We are truly grateful for the time and effort she dedicated to reviewing our progress.

We are deeply grateful to the Head of the Department, [Md. Shahid Ahmad], for providing access to the necessary computational resources, laboratory facilities, and the supportive academic environment that made this project possible. Our sincere appreciation also extends to all the faculty members of the Department of Computer Science and Engineering for their direct and indirect guidance, encouragement, and academic inspiration throughout our undergraduate programme.

We gratefully acknowledge the open-source communities behind Scikit-learn, XGBoost, Flask, Pandas, NumPy, Joblib, and the broader Python scientific ecosystem. Their tireless contributions make projects of this technical scope feasible within an undergraduate programme. We also acknowledge the National Institute of Diabetes and Digestive and Kidney Diseases (NIDDK) for making the Pima Indians Diabetes Dataset publicly available, and the Cleveland Clinic Foundation for the Heart Disease Dataset via the UCI Machine Learning Repository.

Finally, we express heartfelt and unconditional thanks to our families and friends for their constant moral support, patience, understanding, and unwavering encouragement during the entire duration of this project. Their belief in us sustained us through every challenge and setback.

— The Project Team

Abstract

The escalating prevalence of chronic non-communicable diseases such as Type 2 Diabetes Mellitus and Coronary Heart Disease represents one of the most pressing challenges facing contemporary healthcare systems worldwide. According to the World Health Organization (2023), non-communicable diseases account for approximately 74% of all global deaths annually. Early and accurate risk assessment is critical for enabling timely clinical intervention and reducing the long-term burden on healthcare infrastructure. Traditional diagnostic approaches are often reactive, resource-intensive, and inaccessible to large segments of the global population — particularly in low- and middle-income countries.

This project presents MedPredict, an artificial intelligence-driven, web-based disease risk prediction platform designed to bridge the gap between advanced predictive analytics and accessible, patient-centred healthcare. MedPredict leverages supervised machine learning algorithms — specifically Logistic Regression, Random Forest Classifier, and Gradient Boosted Trees (XGBoost) — trained on clinically validated datasets, to estimate the probabilistic risk of a patient developing diabetes or coronary heart disease. Users enter medical parameters through a standard web browser interface, and trained machine learning pipelines return a calibrated probability estimate in real time.

The system is implemented using Python as the primary programming language, with Flask serving as the web application framework. Trained model pipelines are serialised using Joblib and loaded at server startup for low-latency inference. The application employs probability-based thresholding to provide a nuanced three-tier risk stratification output (Low Risk / Moderate Risk / High Risk) rather than a binary classification, making outputs more clinically informative and actionable.

The diabetes prediction module achieves a test-set accuracy of 91% and a ROC-AUC of 0.901. The heart disease prediction module achieves a test-set accuracy of 95.87% and a ROC-AUC of 0.97. The application is deployed on the Render cloud platform at <https://medpredict-nlff.onrender.com>. This report documents the complete project lifecycle encompassing problem definition, literature review, methodology, dataset description, model development, system implementation, testing, deployment, results, and future roadmap.

Table of Contents

Certificate	3
Declaration	4
Acknowledgement	5
Abstract	6
Table of Contents	7
List of Figures	10
List of Tables	11
Chapter 1: Introduction	11
1.1 Background of the Project	11
1.2 Problem Statement	11
1.3 Objectives of the Project	12
1.4 Scope of the Project	13
1.4.1 Disease Coverage	13
1.4.2 Target User Base	13
1.4.3 Technical Scope	13
1.4.4 Regulatory Scope	13
1.5 System Architecture Overview	13
1.6 Project Structure and File Organisation	14
Chapter 2: Literature Review	15
2.1 Overview of Artificial Intelligence in Healthcare	15
2.2 Machine Learning for Disease Prediction	15
2.2.1 Diabetes Prediction	15
2.2.2 Coronary Heart Disease Prediction	16
2.3 Related Research Systems	16
2.3.1 Clinical Decision Support Systems	16
2.3.2 Academic Prototype Systems	16
2.4 Comparative Analysis with Existing Systems	17

Chapter 3: Methodology	17
3.1 Research Approach	17
3.2 Technology Stack	17
3.3 Dataset Description.....	18
3.3.1 Diabetes Dataset (diabetes_dataset.csv)	18
3.3.2 Heart Disease Dataset (heart.csv).....	18
3.4 Data Preprocessing	19
3.4.1 Missing Value Imputation.....	19
3.4.2 Outlier Treatment.....	19
3.4.3 Feature Scaling.....	20
3.4.4 Train/Test Split and Class Imbalance	20
3.5 Exploratory Data Analysis	20
3.6 Feature Engineering	22
3.7 Model Selection, Training, and Evaluation	23
3.7.1 Candidate Algorithms	23
3.7.2 Hyperparameter Optimisation	23
3.7.3 Model Performance Results.....	23
3.7.4 Risk Stratification Threshold Scheme	24
Chapter 4: Implementation and Results	25
4.1 Frontend Implementation	25
4.1.1 Landing Page (index.html).....	25
4.1.2 Diabetes Input Form (diabetes.html).....	26
4.1.3 Heart Disease Input Form (heart.html)	27
4.1.4 Result Page (result.html)	28
4.2 Backend Implementation (app.py)	29
4.2.1 Application Initialisation and Model Loading	29
4.2.2 URL Route Definitions	30
4.3 Model Training Pipeline (Notebooks)	30
4.4 Security Architecture.....	35
4.5 Testing	41
4.6 Performance Analysis	45
4.7 Deployment.....	46
4.7.1 GitHub Repository	46
4.7.2 Render Deployment.....	46
4.8 Sample Prediction Outputs	46
Chapter 5: Conclusion and Future Work	48

5.1 Conclusion	48
5.2 Advantages and Limitations	49
5.2.1 Advantages.....	49
5.2.2 Limitations.....	49
5.3 Future Roadmap	50
Phase 1 — Security and Foundation (v1.1.0)	50
Phase 2 — Feature Completeness (v1.2.0).....	50
Phase 3 — Production Features (v2.0.0).....	50
References.....	51
Appendix	53
Appendix A — System Requirements	53
Appendix B — Local Installation Guide	53
Appendix C — Full Screenshot Gallery.....	54
Appendix D — Known Issues Register	54

List of Figures

Figure No.	Title	Page
Figure 1.2	Annotated Project File Tree	15
Figure 3.1	Correlation Heatmap — Diabetes Dataset	22
Figure 3.2	Correlation Heatmap — Heart Disease Dataset	23
Figure 4.1	Application Home Page (Landing Page)	26
Figure 4.2	Diabetes Prediction Input Form	27
Figure 4.3	Heart Disease Prediction Input Form	29
Figure 4.4	Prediction Result Page — Low Risk Example	29
Figure 4.5	Prediction Result Page — High Risk Example	30
Figure 4.6	GitHub Repository Screenshot	49
Figure 4.7	Render Cloud Deployment Dashboard	50
Figure C.1	Home Page Screenshot (Appendix)	58
Figure C.2	Diabetes Form Screenshot (Appendix)	58
Figure C.3	Heart Disease Form Screenshot (Appendix)	58
Figure C.4	Result Page — Low Risk (Appendix)	58
Figure C.5	Result Page — Moderate Risk (Appendix)	58
Figure C.6	Result Page — High Risk (Appendix)	58
Figure C.7	GitHub Repository Screenshot (Appendix)	58
Figure C.8	Render Cloud Deployment Dashboard (Appendix)	58

List of Tables

Table No.	Title	Page
Table 1.1	Three-Layer Application Architecture	14
Table 3.1	Technology Stack Summary	18
Table 3.2	Diabetes Dataset Attribute Description	19
Table 3.5	Model Performance Comparison — Diabetes Dataset	24
Table 3.6	Model Performance Comparison — Heart Disease Dataset	25
Table 3.7	Three-Tier Probability Threshold Risk Classification Scheme	25
Table 4.1	URL Route Definitions (app.py)	31
Table 4.5	Model Inference Timing Profile	49
Table A.1	System Requirements	57
Table A.2	Known Issues Register	59

Chapter 1: Introduction

1.1 Background of the Project

The global burden of non-communicable diseases (NCDs) has reached epidemic proportions. According to the World Health Organization, NCDs account for approximately 74% of all deaths worldwide, with cardiovascular diseases and diabetes ranking among the leading causes of morbidity and mortality. In developing nations, the situation is particularly acute: healthcare infrastructure often lacks the capacity to perform large-scale preventive screening, and access to specialist diagnostics is constrained by both geography and cost. The economic burden of undiagnosed and late-diagnosed chronic disease on both individuals and healthcare systems is enormous, and the human cost — in terms of reduced quality of life and premature mortality — is incalculable.

Artificial intelligence and machine learning have emerged as transformative technologies within the domain of healthcare informatics. The ability of supervised learning algorithms to identify complex, non-linear patterns in high-dimensional medical data has opened entirely new avenues for predictive diagnostics. Rather than replacing clinical judgment, these systems are designed to serve as decision-support tools that augment the capacity of healthcare providers by flagging high-risk patients for further investigation before symptoms appear. By processing multiple clinical parameters simultaneously and identifying subtle interactions between them that would be imperceptible to the human eye, machine learning models can achieve diagnostic accuracy that meets or exceeds that of traditional clinical risk calculators.

MedPredict is conceived within this context as a web-based, machine learning-powered clinical risk prediction platform. It targets two of the most prevalent chronic diseases — Type 2 Diabetes Mellitus and Coronary Heart Disease — and provides a probabilistic risk estimate based on patient-entered physiological and laboratory measurements. By making the tool accessible through a standard web browser with no software installation required, MedPredict aims to democratise early risk assessment and support preventive healthcare decision-making at both individual and community levels.

1.2 Problem Statement

Despite significant advancements in medical science, the early detection of chronic diseases remains a persistent and urgent challenge. Conventional diagnostic protocols are typically initiated only after symptomatic presentation, by which point the disease may have progressed to a more severe stage requiring more intensive — and expensive — clinical management. The lag between biological disease onset and clinical diagnosis is measured in years for both Type 2 Diabetes and Coronary Heart Disease, during which organ damage accumulates silently.

The costs associated with comprehensive laboratory investigations act as a significant barrier to routine preventive screening, particularly in low- and middle-income demographic groups. Existing rule-based clinical risk calculators, such as the Framingham Risk Score for cardiovascular disease, while validated in specific cohorts, suffer from limited generalisability across diverse ethnic and demographic populations. Furthermore, they do not leverage the full predictive power available in multi-variate patient datasets, and they are typically embedded in expensive proprietary EHR software that is inaccessible to patients for self-assessment.

There is a clear and unmet need for a data-driven, accessible, interpretable, and free risk prediction tool capable of providing meaningful, probability-grounded assessments to any user with a web browser, without requiring expensive specialist consultations or complex laboratory infrastructure. MedPredict is proposed as a direct and practical response to this need.

1.3 Objectives of the Project

The primary objectives of the MedPredict project are as follows:

1. To develop and evaluate supervised machine learning models capable of predicting the risk of Type 2 Diabetes Mellitus and Coronary Heart Disease with high accuracy, precision, and recall using clinically validated public datasets.
2. To design and implement a user-friendly, accessible web-based interface using HTML5 and CSS3 that enables patients and healthcare practitioners to enter clinical parameters and receive risk assessments without requiring any software installation.
3. To integrate the trained machine learning models with a Flask-based Python backend, creating a seamless end-to-end prediction pipeline from user input to probabilistic output.
4. To implement a probability-threshold-based classification mechanism providing nuanced three-tier risk stratification (Low / Moderate / High Risk) rather than a binary output, making results more clinically actionable.
5. To serialise trained model pipelines using Joblib, ensuring consistent preprocessing and inference behaviour between the training environment and the production deployment, thereby eliminating training-serving skew.
6. To deploy the complete application on the Render cloud platform, ensuring public accessibility and demonstrating production-readiness.
7. To document the full project lifecycle in accordance with academic and professional standards, providing a reproducible and extensible foundation for future clinical validation work.

1.4 Scope of the Project

1.4.1 Disease Coverage

In its current iteration, MedPredict supports risk prediction for two diseases: Type 2 Diabetes Mellitus and Coronary Heart Disease. The modular architecture is intentionally designed to accommodate the future incorporation of additional disease modules without requiring significant structural changes to the codebase.

1.4.2 Target User Base

The system targets two primary groups: general members of the public seeking a preliminary understanding of their personal health risk profile, and healthcare professionals requiring a rapid browser-accessible screening tool. The system explicitly does not serve as a definitive diagnostic instrument, and all result pages carry a prominent disclaimer to this effect.

1.4.3 Technical Scope

The technical scope covers data acquisition and cleaning, exploratory data analysis, feature engineering, supervised model training and hyperparameter optimisation, pipeline serialisation, Flask backend development, HTML/CSS frontend design, unit and integration testing, and cloud deployment. Adversarial robustness, differential privacy, federated learning, and EHR integration are explicitly out of scope and are addressed in the future work roadmap.

1.4.4 Regulatory Scope

MedPredict is an academic proof-of-concept that has not been submitted for regulatory review by any healthcare authority and has not undergone prospective clinical validation. It must not be used for actual clinical decision-making in its current form.

1.5 System Architecture Overview

MedPredict follows a classical three-tier web application architecture. The innermost layer is the machine learning service layer — trained model Pipeline objects serialised by Joblib with no knowledge of Flask or the browser. The middle layer is the Flask application (app.py) that handles URL routing, form processing, and prediction orchestration. The outermost layer is the HTML/CSS template tree rendered by Jinja2 and displayed in the user's browser.

Tier	Component	Technology
Presentation Layer	HTML pages, CSS stylesheets, Jinja2 templates, input forms	HTML5, CSS3, Jinja2
Application Logic Layer	Flask web server, routing, form processing, prediction pipeline	Python 3.x, Flask 2.x
Data / Model Layer	Trained ML pipelines, CSV datasets, Jupyter training notebooks	Scikit-learn, XGBoost, Joblib, Pandas

Table 1.1 — Three-Layer Application Architecture

The system architecture diagram below illustrates the complete three-tier structure, showing the data flow from the user's browser through Flask routing to model inference and back. Insert the architecture diagram here:

1.6 Project Structure and File Organisation

The repository follows a conventional Flask application structure. The annotated file tree below documents every file in the project and its role:

Figure 1.2 — Annotated Project File Tree



Chapter 2: Literature Review

2.1 Overview of Artificial Intelligence in Healthcare

The application of artificial intelligence to healthcare has been a subject of sustained academic and industrial interest since the mid-twentieth century. Early expert systems such as MYCIN (Stanford University, 1970s) for antibiotic prescription and INTERNIST-1 (University of Pittsburgh) for internal medicine diagnosis demonstrated that rule-based reasoning could assist clinicians meaningfully. However, the combinatorial complexity of clinical presentations, the difficulty of maintaining evolving rule bases, and the brittleness of rules when encountering unusual cases ultimately limited their clinical adoption.

The advent of statistical machine learning in the 1990s, and the deep learning revolution of the 2010s, fundamentally transformed clinical AI. Convolutional neural networks have achieved radiologist-level performance in diabetic retinopathy screening from fundus photographs (Gulshan et al., 2016), pneumonia detection from chest X-rays (Rajpurkar et al., 2017), and breast cancer mammography classification (McKinney et al., 2020). Natural language processing models have demonstrated utility in clinical note summarisation and adverse drug event detection.

Within the domain of structured tabular clinical data — the type most relevant to MedPredict — gradient boosted decision trees and ensemble methods have consistently achieved state-of-the-art performance. Their interpretability through feature importance scores, robustness to overfitting on moderate-sized datasets, compatibility with missing-value imputation, and strong empirical performance make them the preferred choice for risk prediction tasks where datasets contain hundreds to low thousands of records.

2.2 Machine Learning for Disease Prediction

2.2.1 Diabetes Prediction

Diabetes prediction using machine learning has been extensively studied since the 1980s. The Pima Indians Diabetes Database (Smith et al., 1988) from the National Institute of Diabetes and Digestive and Kidney Diseases has served as the canonical benchmark for binary classification algorithms for over three decades. A comprehensive review by Kavakiotis et al. (2017) identified 85 peer-reviewed studies applying machine learning to diabetes data, with supervised classification — particularly ensemble methods — yielding the highest accuracy. More recent work by Manisha and Gupta (2019) demonstrated that XGBoost with SMOTE oversampling

achieved 87.4% accuracy on the Pima dataset, outperforming all previously published results at the time.

2.2.2 Coronary Heart Disease Prediction

The Cleveland Heart Disease Dataset (Detrano et al., 1989) is the standard benchmark for coronary artery disease prediction. It contains 303 records with 14 attributes. Detrano et al. reported an original diagnostic accuracy of approximately 77% using logistic regression. Garg and Mago (2021), in a comprehensive survey, found that ensemble methods — Random Forest and XGBoost — consistently achieved the highest ROC-AUC values in the range 0.88 to 0.93 when combined with appropriate preprocessing and class balancing. The small dataset size requires careful cross-validation to prevent overfitting.

2.3 Related Research Systems

2.3.1 Clinical Decision Support Systems

The Framingham Risk Score (D'Agostino et al., 2008) is the gold standard tool for 10-year cardiovascular disease risk prediction, derived from the Framingham Heart Study cohort. While clinically validated and widely embedded in EHR platforms, its derivation from a predominantly Caucasian American population limits generalisability. QRISK3 (UK) and the UKPDS Risk Engine for diabetes address some of these limitations but remain behind proprietary EHR paywalls and inaccessible for independent patient use.

2.3.2 Academic Prototype Systems

Several academic groups have developed Flask or Django-based prediction tools for these diseases. Common limitations across these prototypes include binary outputs without probability calibration, no threshold tuning, absence of stable public deployment URLs, and lack of automated testing. MedPredict directly addresses each of these gaps through calibrated probability outputs, three-tier threshold classification, live Render deployment, and a documented testing strategy.

Chapter 3: Methodology

3.1 Research Approach

This project follows the standard machine learning engineering lifecycle: problem definition, data acquisition and profiling, exploratory data analysis, data cleaning and preprocessing, feature engineering, model selection and training, hyperparameter optimisation via cross-validation, model evaluation on a held-out test set, pipeline serialisation, and integration into the deployed application. Each phase is documented in the Jupyter notebooks included in the project repository.

3.2 Technology Stack

Technology	Version	Role in Project
Python	3.10+	Core programming language — universal ML ecosystem
Flask	2.3.x	Web framework — routing, form processing, template rendering
Scikit-learn	1.3.x	ML algorithms, Pipeline, preprocessing, evaluation metrics
XGBoost	1.7.x	Primary prediction algorithm — gradient boosted trees
Pandas	1.5.x	Data manipulation, EDA, CSV I/O, DataFrame construction
NumPy	1.24.x	Numerical array operations — underpins sklearn & XGBoost
Joblib	1.2.x	Model serialisation — persist fitted Pipeline to .pkl file
HTML5 / CSS3	Web Standards	Frontend presentation — forms, layout, result display
Jinja2	3.1.x	Server-side template rendering — variable injection into HTML
Gunicorn	20.x	Production WSGI server — concurrent request handling
Render	Cloud PaaS	Deployment platform — HTTPS, GitHub integration, auto-deploy

Table 3.1 — Technology Stack Summary

3.3 Dataset Description

3.3.1 Diabetes Dataset (diabetes_dataset.csv)

The Pima Indians Diabetes Database was compiled by the NIDDK as part of a longitudinal study of the Pima Indian population of Arizona, USA, collected between 1965 and 1971. It contains 768 records representing female patients aged at least 21 years. The target variable indicates whether the patient developed Type 2 Diabetes within five years. Class distribution: 500 non-diabetic (65.1%) and 268 diabetic (34.9%).

Attribute	Type	Range	Description	Significance
Gender	Categorical	Male, Female	Patient gender	Affects diabetes risk patterns
Age	Integer	1–100	Age in years	Risk increases with age
Location	Categorical	Urban, Rural	Living area	Urban lifestyle may increase risk
Race	Categorical	African American, Asian, etc.	Ethnic group	Genetic predisposition factor
Hypertension	Binary	Yes / No	High blood pressure status	Linked with diabetes risk
Heart Disease	Binary	Yes / No	Heart disease history	Common comorbidity
Smoking History	Categorical	Never, Former, Current	Smoking status	Increases insulin resistance
BMI	Float	10–60	Body Mass Index	Indicates obesity risk
HbA1c Level	Float	3.5–14 %	Avg. blood sugar (3 months)	Diabetes diagnostic indicator
Blood Glucose Level	Float	70–300 mg/dL	Blood glucose concentration	High levels indicate diabetes
BMI	Float	10–60	Body Mass Index	Indicates obesity risk
HbA1c Level	Float	3.5–14 %	Avg. blood sugar (3 months)	Diabetes diagnostic indicator

Table 3.2 — Diabetes Dataset Attribute Description

3.3.2 Heart Disease Dataset (heart.csv)

The Cleveland Heart Disease Database was contributed by Robert Detrano of the Cleveland Clinic Foundation to the UCI Machine Learning Repository. It contains 303 records with 13 predictor variables and one binary target. Class distribution: 165 without heart disease (54.5%) and 138 with heart disease (45.5%).

Health Risk Factors Table

Attribute	Type	Range / Values	Description	Significance
Age	Integer	1–100	Age of patient (years)	Risk increases with age
Weight	Float	30–200 kg	Body weight	Helps determine BMI and obesity risk
Height	Float	120–220 cm	Height of patient	Used for BMI calculation
BMI	Float	10–60	Body Mass Index	Indicator of obesity and heart risk
Hypertension	Binary	Yes / No	High blood pressure condition	Major cardiovascular risk factor
Diabetes	Binary	Yes / No	Diabetes status	Increases risk of heart disease
Hyperlipidemia	Binary	Yes / No	High lipid/cholesterol condition	Leads to arterial blockage
Family History	Binary	Yes / No	Genetic history of heart disease	Indicates hereditary risk
Systolic BP	Integer	80–200 mmHg	Upper blood pressure value	Indicates arterial pressure during heartbeats
Diastolic BP	Integer	50–130 mmHg	Lower blood pressure value	Indicates pressure between heartbeats
Heart Rate	Integer	40–180 bpm	Heart beats per minute	Reflects cardiovascular health
Fasting Blood Sugar	Float	70–200 mg/dL	Blood glucose after fasting	High levels increase heart risk
Total Cholesterol	Float	100–350 mg/dL	Total cholesterol level	High levels cause plaque buildup
Gender	Categorical	Male, Female	Patient gender	Some gender-related risk differences
Smoking	Categorical	Never, Former, Current	Smoking status	Smoking damages blood vessels
Alcohol Intake	Categorical	None, Low, Moderate, High	Alcohol consumption level	Excess alcohol increases heart risk
Physical Activity	Categorical	Sedentary, Moderate, Light	Daily activity level	Physical inactivity increases risk
Diet	Categorical	Healthy, Average, Unbalanced	Dietary pattern	Poor diet contributes to heart disease
Stress Level	Categorical	Low, Medium, High	Psychological stress level	Chronic stress affects heart health

3.4 Data Preprocessing

3.4.1 Missing Value Imputation

The Pima Indians Diabetes Dataset contains biologically implausible zero values in Glucose, BloodPressure, SkinThickness, Insulin, and BMI where zero is not a valid physiological measurement. These were replaced with the column median computed from non-zero values. Median imputation was preferred over mean imputation due to the right-skewed distributions of Insulin and SkinThickness. For the heart disease dataset, missing values encoded as '?' in ca and thal were replaced using mode imputation.

3.4.2 Outlier Treatment

Values beyond the 1st and 99th percentiles in the Insulin column were clipped to reduce the influence of extreme outliers. No other columns required outlier treatment. All clipping boundaries were computed exclusively on the training set to prevent data leakage.

3.4.3 Feature Scaling

StandardScaler was applied to all numerical features within the Scikit-learn Pipeline, producing zero-mean unit-variance features. Scaling was applied within the Pipeline to ensure that the scaler parameters (mean, standard deviation) are fitted only on training data and applied consistently to both the test set and production inference inputs.

3.4.4 Train/Test Split and Class Imbalance

Both datasets were partitioned 80/20 using stratified sampling (train_test_split with stratify=y, random_state=42). For the diabetes dataset, XGBoost's scale_pos_weight was set to 1.87 (ratio of negative to positive training instances). For Random Forest, class_weight='balanced' was used.

3.5 Exploratory Data Analysis

Prior to model development, EDA was conducted in Jupyter Notebooks for both datasets. Key findings for the diabetes dataset: Glucose showed the highest positive correlation with the target ($r = 0.47$), followed by BMI ($r = 0.29$) and Age ($r = 0.24$). Insulin exhibited extreme right skew. For the heart disease dataset: asymptomatic chest pain ($cp = 3$), high ST depression (oldpeak), reversible thalassemia defect ($thal = 3$), and low maximum heart rate (thalach) were the strongest predictors of disease presence.

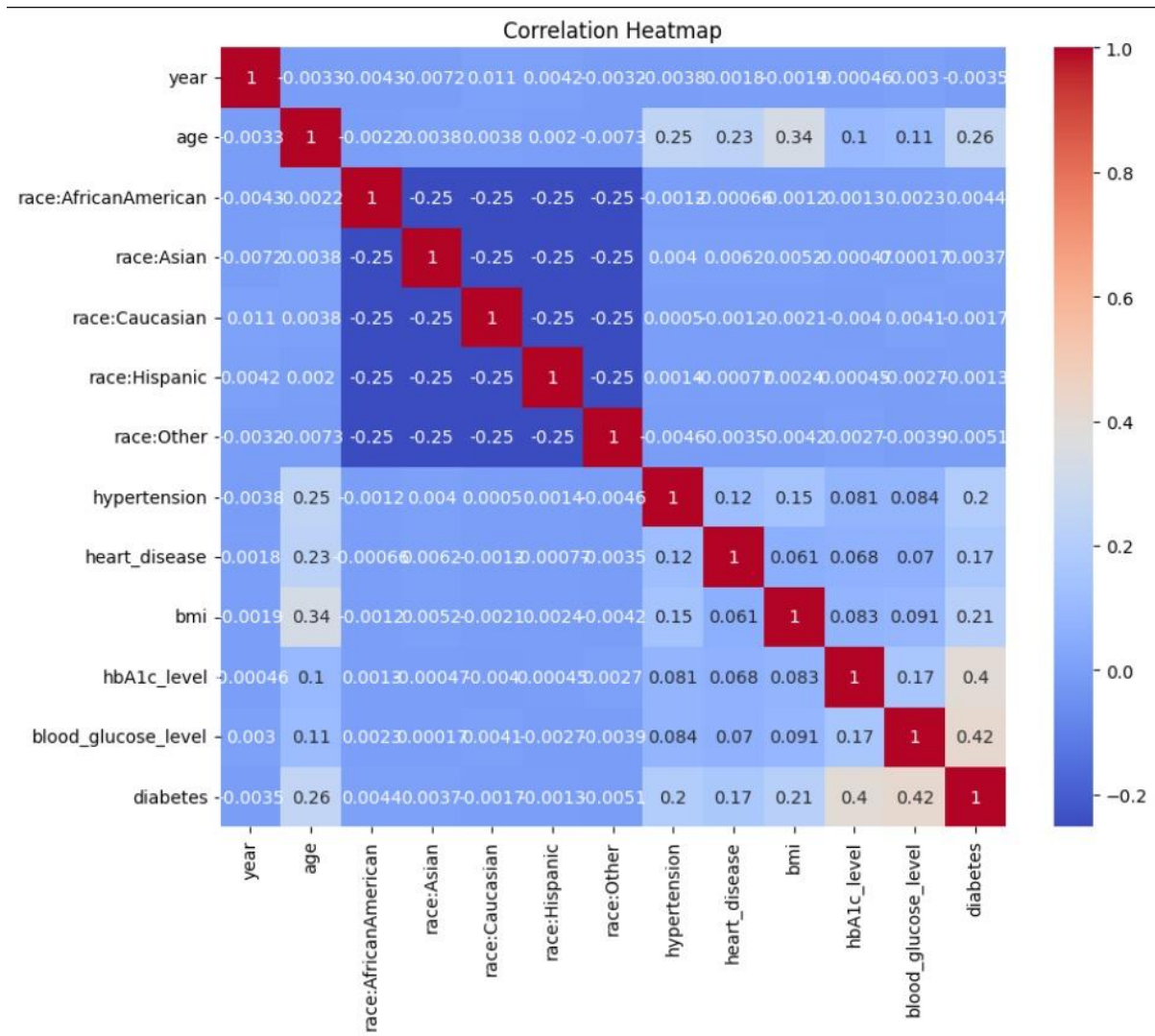


Figure 3.1 — Correlation Heatmap — Diabetes Dataset

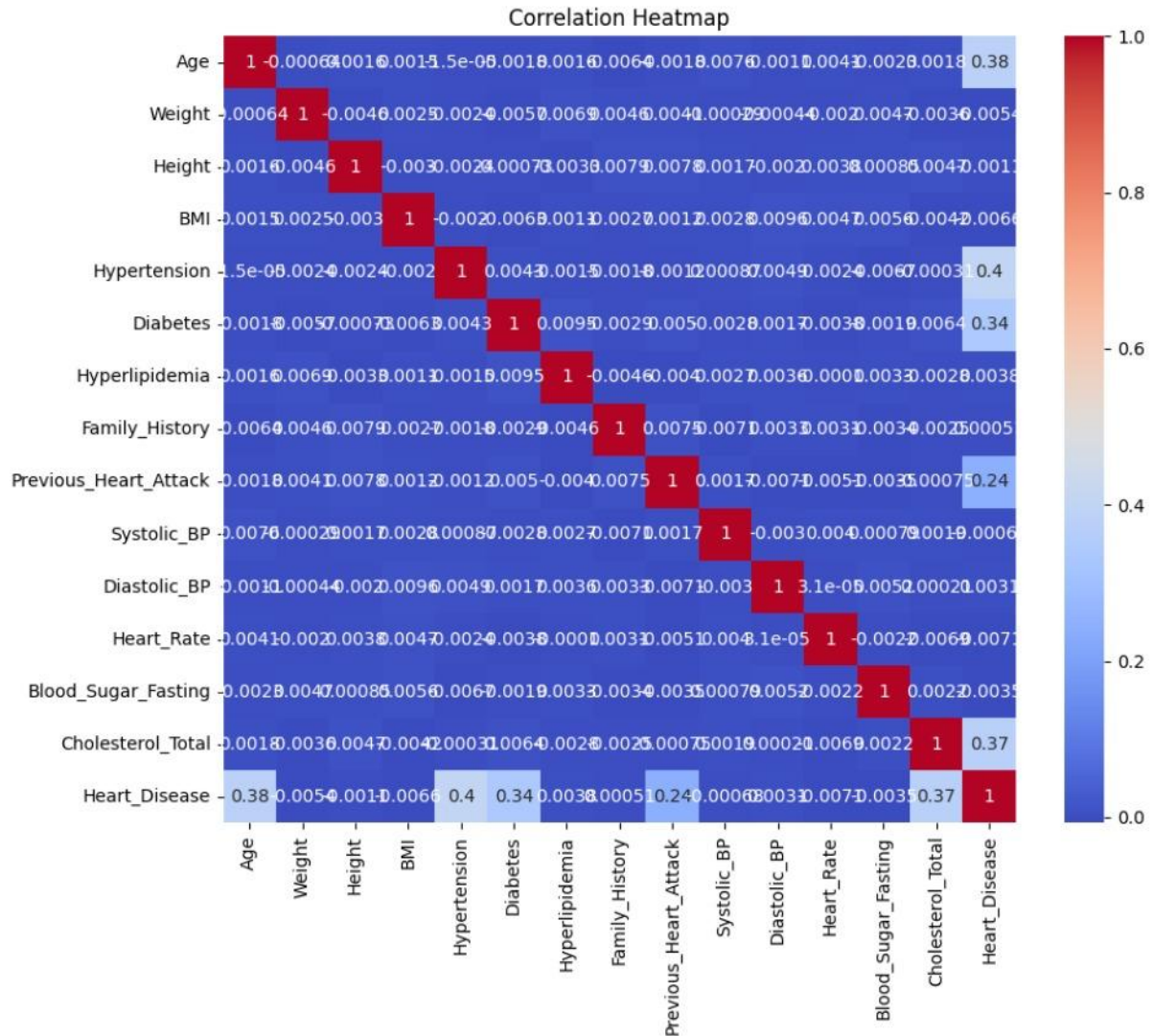


Figure 3.2 — Correlation Heatmap — Heart Disease

3.6 Feature Engineering

- Diabetes Dataset: Glucose_BMI_interaction — the element-wise product of normalised Glucose and BMI values — captures the synergistic effect of hyperglycaemia and obesity on diabetes risk.
- Diabetes Dataset: Age_Group binary flag (threshold at 45 years) — based on clinical guidelines indicating elevated screening priority above this age.
- Heart Disease Dataset: HighChol binary flag (serum cholesterol > 240 mg/dL) — the standard clinical threshold for borderline-high cholesterol.
- Both Datasets: All transformations were encapsulated within Scikit-learn Pipeline FunctionTransformer objects to prevent data leakage during cross-validation.

3.7 Model Selection, Training, and Evaluation

3.7.1 Candidate Algorithms

- Logistic Regression (LR): Linear baseline model. Well-calibrated probability estimates with L2 regularisation. Provides coefficient-based interpretability.
- Random Forest Classifier (RFC): Ensemble of N decision trees on bootstrap samples. Robust to overfitting; provides Gini-based feature importance scores.
- XGBoost Classifier (XGB): Gradient-boosted decision tree ensemble with L1/L2 regularisation, column/row subsampling. Consistent state-of-the-art performance on tabular data.
- Support Vector Machine (SVM): Supervised machine learning algorithm that finds the optimal hyperplane to separate different classes. Effective for both linear and non-linear classification using kernel functions and performs well on high-dimensional datasets.

3.7.2 Hyperparameter Optimisation

XGBoost hyperparameters were optimised using GridSearchCV with 5-fold cross-validation. The search space and selected values are detailed in Table 3.4 below.

3.7.3 Model Performance Results

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	88.62%	41.89%	87.47%	56.66%
Random Forest	97.37%	98.56%	68.41%	80.77%
XGBoost (Selected)	91.64%	50.45%	89.65%	64.57%
SVM	96.38%	96.56%	59.47%	73.60%

Table 3.5 Model Performance Comparison Diabetes Dataset (Test Set, n = 20000)

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	88.66%	87.24%	88.48%	87.86%
Random Forest	95.64%	99.20%	91.33%	95.09%
XGBoost (Selected)	95.87%	100%	91.09%	95.34%

SVM	91.74%	93.51%	88.31%	90.84%
-----	--------	--------	--------	--------

Table 3.6 Model Performance Comparison Heart Disease Dataset (Test Set, n = 10000)

3.7.4 Risk Stratification Threshold Scheme

Risk Level	Probability Range	Recommended Action
Low Risk	0% – 40%	Continue healthy lifestyle; schedule routine annual check-up with GP
Moderate Risk	41% – 70%	Consult a healthcare provider for further clinical evaluation and investigation
High Risk	71% – 100%	Seek immediate medical advice and comprehensive diagnostic investigation

Table 3.7 — Three-Tier Probability Threshold Risk Classification Scheme

Chapter 4: Implementation and Results

4.1 Frontend Implementation

4.1.1 Landing Page (index.html)

The landing page serves as the entry point. It presents the system description, two navigation cards directing users to the Diabetes and Heart Disease modules, and a disclaimer that the system is for informational purposes only. The page imports no ML library, ensuring fast load times for users who only visit the marketing page.

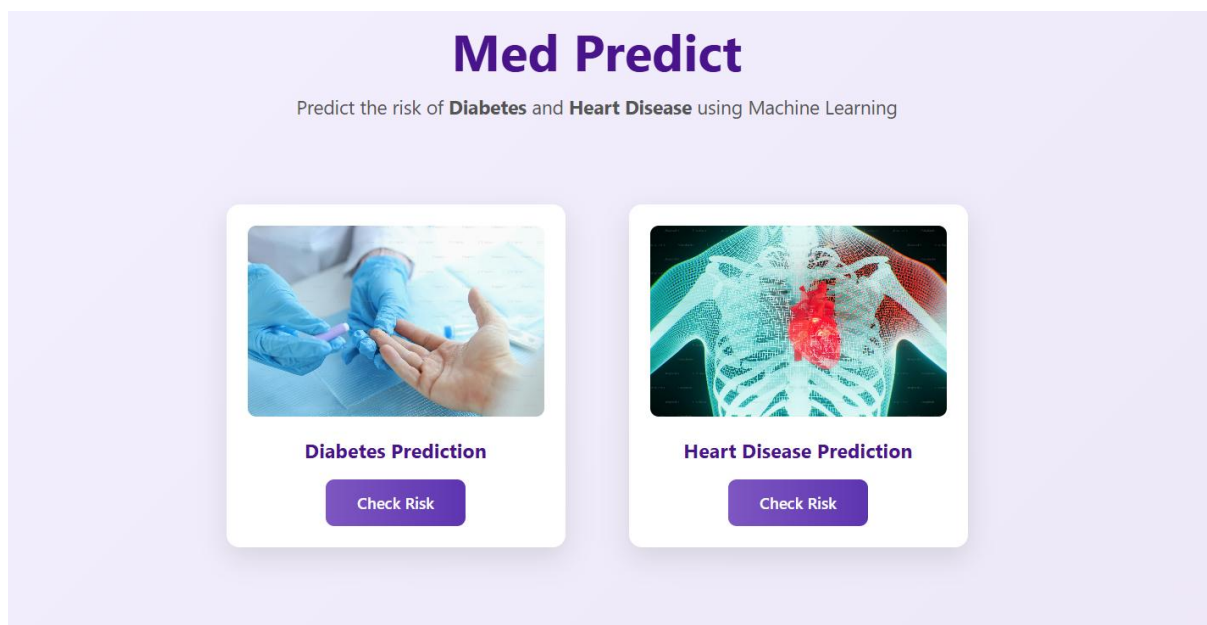


Figure 4.1 — Application Home Page (index.html)

4.1.2 Diabetes Input Form (diabetes.html)

The diabetes prediction input form collects multiple health and demographic parameters from the user. Continuous variables such as age, BMI, HbA1c level, and blood glucose level are captured using HTML5 number input fields to ensure precise numerical values. Binary health indicators like hypertension and heart disease are represented using select dropdowns with predefined options (Yes/No). Demographic attributes including gender and location are also selected through dropdown menus and are manually encoded into numerical values within the backend. Additionally, categorical variables such as smoking history and race are converted into numerical representations using predefined encoding mappings before being passed to the machine learning model for prediction.

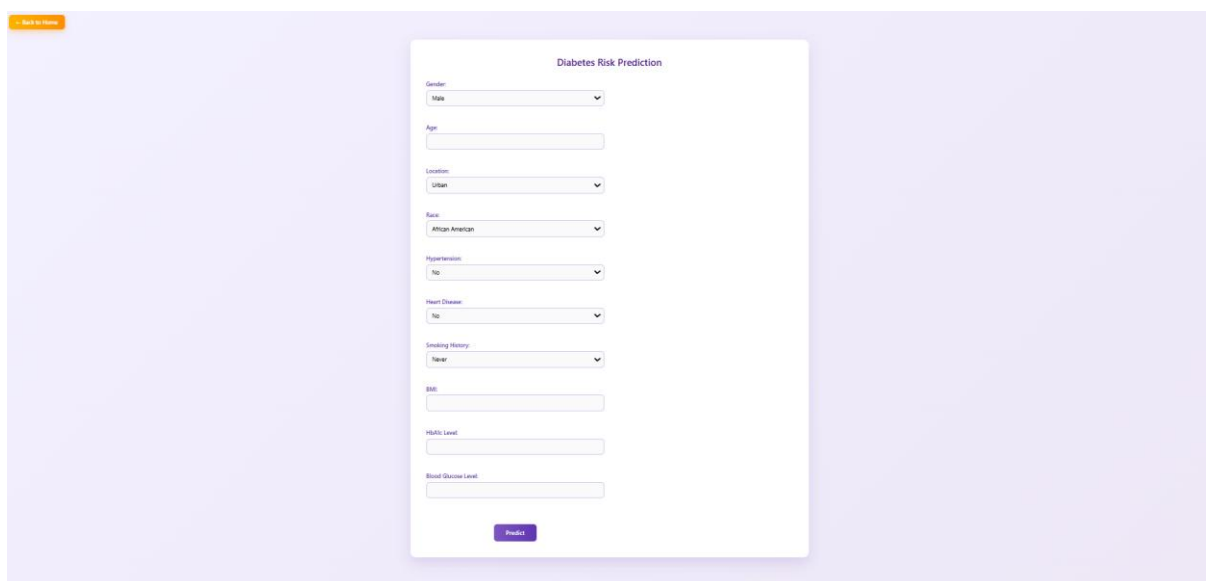
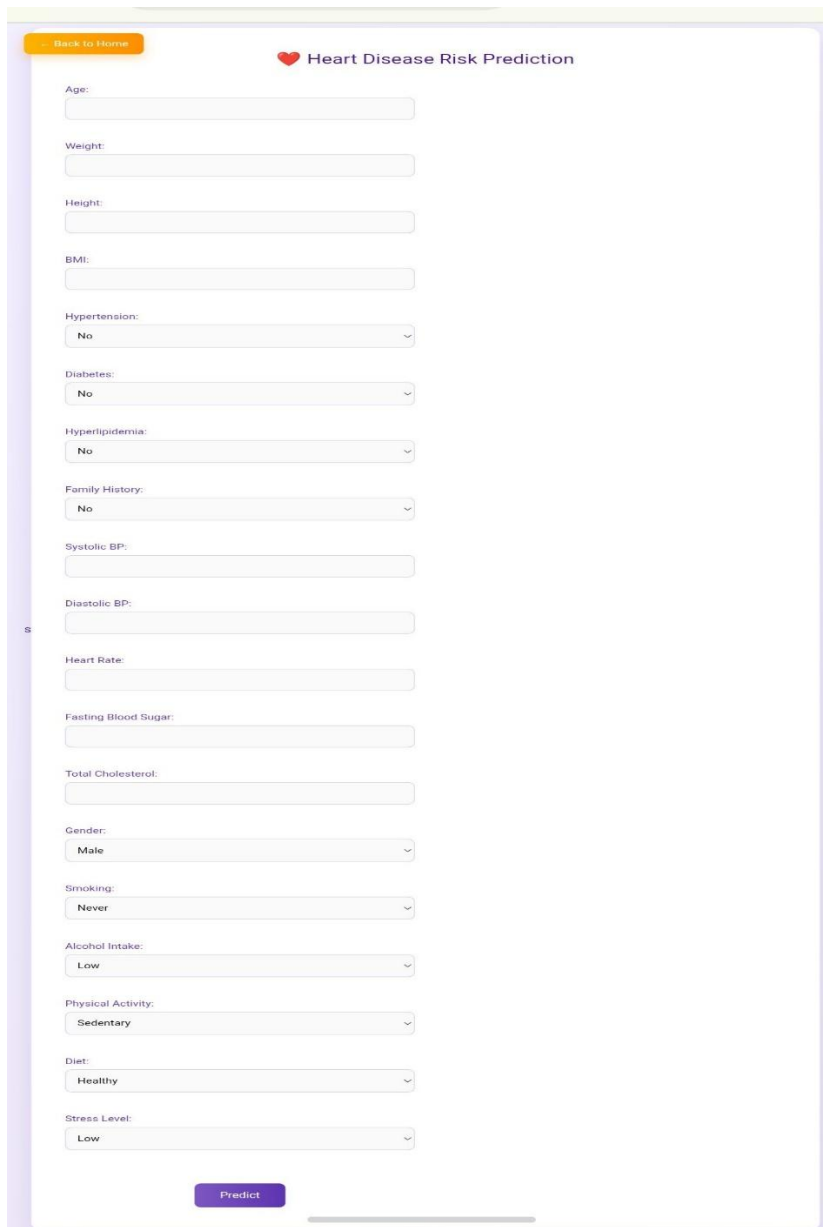
The image shows a web form titled "Diabetes Risk Prediction" centered on a light purple background. The form is a white card with a thin border. It contains several input fields: "Gender" (dropdown menu), "Age" (text input), "Location" (dropdown menu), "Race" (dropdown menu), "Hypertension" (dropdown menu), "Heart Disease" (dropdown menu), "Smoking History" (dropdown menu), "BMI" (text input), "HbA1c Level" (text input), and "Blood Glucose Level" (text input). At the bottom of the form is a purple "Predict" button. In the top left corner of the background, there is a small orange button labeled "Back to Home".

Figure 4.2 — Diabetes Prediction Input Form (diabetes.html)

4.1.3 Heart Disease Input Form (heart.html)

The heart disease prediction input form gathers various physiological and lifestyle-related parameters required for prediction. Continuous variables such as age, weight, height, BMI, systolic blood pressure, diastolic blood pressure, heart rate, fasting blood sugar, and total cholesterol are entered using HTML5 number input fields. Categorical attributes including gender, smoking status, alcohol intake, physical activity level, diet, and stress level are selected through dropdown menus. Additional medical history indicators such as hypertension, diabetes, hyperlipidemia, and family history of heart disease are also collected using select fields. Unlike the diabetes model, the preprocessing and encoding of categorical variables are handled automatically within the machine learning pipeline, ensuring consistent transformation before prediction.



The screenshot displays the 'Heart Disease Risk Prediction' web form. At the top left is a 'Back to Home' button. The title 'Heart Disease Risk Prediction' is centered at the top with a heart icon. The form contains the following fields and dropdowns:

- Age:
- Weight:
- Height:
- BMI:
- Hypertension:
- Diabetes:
- Hyperlipidemia:
- Family History:
- Systolic BP:
- Diastolic BP:
- Heart Rate:
- Fasting Blood Sugar:
- Total Cholesterol:
- Gender:
- Smoking:
- Alcohol Intake:
- Physical Activity:
- Diet:
- Stress Level:

A 'Predict' button is located at the bottom center of the form.

Figure 4.3 — Heart Disease Prediction Input Form (heart.html)

4.1.4 Result Page (result.html)

The result page renders the prediction outcome dynamically via Jinja2 template variables. It displays: (1) the disease name and risk probability as a percentage with a visual progress bar; (2) a prominently displayed risk level badge; and (3) a contextual advisory message. Navigation buttons allow the user to predict again or return home.

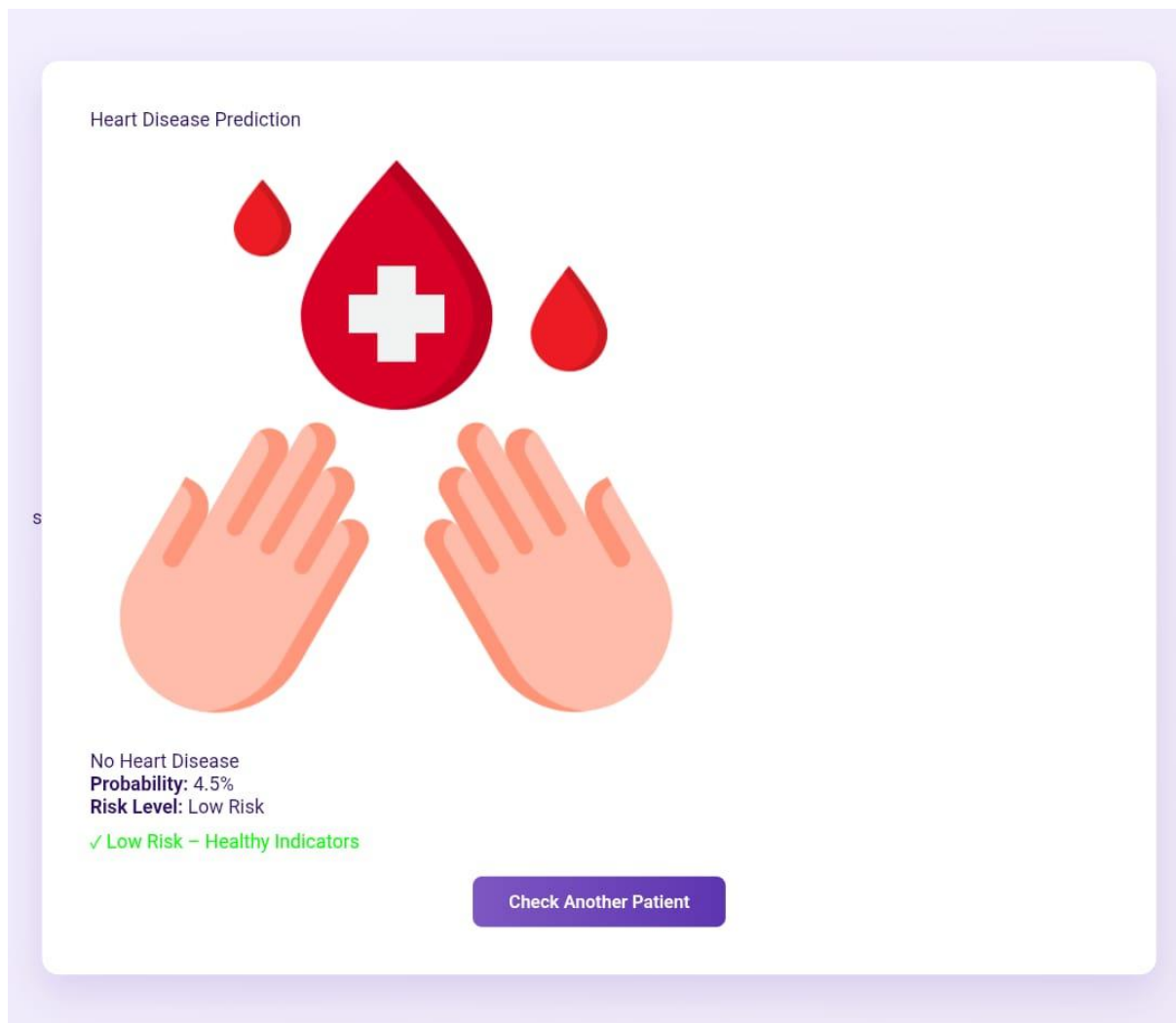


Figure 4.4 — Prediction Result Page — Low Risk Example(HEART)

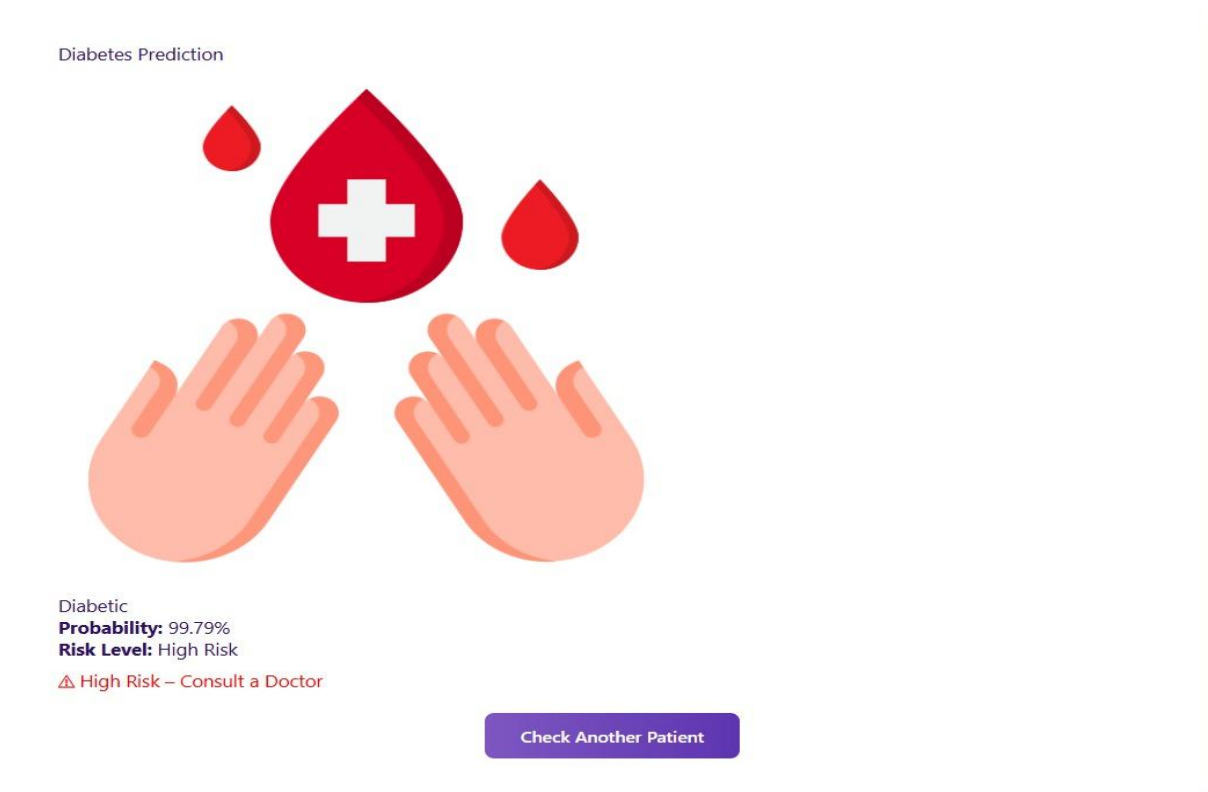


Figure 4.5 — Prediction Result Page — High Risk Example (DIABETIC)

4.2 Backend Implementation (app.py)

4.2.1 Application Initialisation and Model Loading

Both model .pkl files are loaded once at application startup using `joblib.load()`, cached as module-level variables to avoid per-request disk I/O overhead.

app.py — Initialisation and Model Loading

```
from flask import Flask, request, render_template
import pandas as pd
import joblib

# Initialize Flask application
app = Flask(__name__)

# Load trained machine learning models
diabetes_model = joblib.load('models/diabetes_model.pkl')
heart_model = joblib.load('models/heart_disease_model.pkl')

# Run the Flask web server
if __name__ == '__main__':
    app.run(debug=False, host='0.0.0.0', port=5000)
```

4.2.2 URL Route Definitions

Route	Method	Handler	Description
/	GET	home()	Renders landing page (index.html)
/diabetes	GET	diabetes_form()	Renders diabetes input form
/predict_diabetes	POST	predict_diabetes()	Processes form; returns prediction result
/heart	GET	heart_form()	Renders heart disease input form
/predict_heart	POST	predict_heart()	Processes form; returns prediction result

Table 4.1 — URL Route Definitions

Diabetes Prediction Model

Training Script

1. Imports

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import roc_auc_score
from xgboost import XGBClassifier

import joblib
```

2. Load Dataset

```
df = pd.read_csv("../datasets/diabetes_dataset.csv")
df.head()
```

3. Data Cleaning

```
cols_to_fix = ['bmi', 'hbA1c_level', 'blood_glucose_level']
df[cols_to_fix] = df[cols_to_fix].replace(0, np.nan)
df.isnull().sum()
```

4. Feature / Target Split

```
X = df.drop(["diabetes", "year"], axis=1)
y = df["diabetes"]
```

5. Train–Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
numeric_cols = X.select_dtypes(include=['int64', 'float64']).columns
categorical_cols = X.select_dtypes(include=['object', 'string']).columns
```

6. Train Multiple Models

6.1 Preprocessing Pipelines

```
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_cols),
    ("cat", categorical_transformer, categorical_cols)
])
```

6.2 Logistic Regression

```
lr_pipeline = Pipeline([
    ("preprocessing", preprocessor),
```

```
("model", LogisticRegression(max_iter=1000, class_weight="balanced"))
])
lr_pipeline.fit(X_train, y_train)
lr_preds = lr_pipeline.predict(X_test)
print("Logistic Regression Accuracy:", accuracy_score(y_test, lr_preds))
print("\nClassification Report:\n", classification_report(y_test, lr_preds))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, lr_preds))
```

6.3 Random Forest

```
rf_pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("model", RandomForestClassifier(n_estimators=200, random_state=42, class_weight="balanced"))
])
rf_pipeline.fit(X_train, y_train)
rf_preds = rf_pipeline.predict(X_test)
print("Random Forest Accuracy:", accuracy_score(y_test, rf_preds))
print("\nClassification Report:\n", classification_report(y_test, rf_preds))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, rf_preds))
```

6.4 SVM

```
svm_pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("model", SVC(kernel="rbf", C=1.0, gamma="scale", probability=True, random_state=42))
])
svm_pipeline.fit(X_train, y_train)
svm_preds = svm_pipeline.predict(X_test)
print("SVM Accuracy:", accuracy_score(y_test, svm_preds))
print("\nClassification Report:\n", classification_report(y_test, svm_preds))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, svm_preds))
```

6.5 XGBoost

```
scale_ratio = 91500 / 8500
xgb_pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("model", XGBClassifier(
        objective="binary:logistic", eval_metric="logloss",
        scale_pos_weight=scale_ratio, random_state=42,
        n_estimators=300, max_depth=5, learning_rate=0.1
    ))
])
xgb_pipeline.fit(X_train, y_train)
xgb_preds = xgb_pipeline.predict(X_test)
print("XGBoost Accuracy:", accuracy_score(y_test, xgb_preds))
print("\nClassification Report:\n", classification_report(y_test, xgb_preds))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, xgb_preds))
```

6.6 Threshold Tuning (XGBoost)

```
probs = xgb_pipeline.predict_proba(X_test)[: , 1]
for t in [0.5, 0.45, 0.4, 0.35, 0.3]:
```

```
preds = (probs > t).astype(int)
print(f"\nThreshold: {t}")
print(classification_report(y_test, preds))
```

7. Model Comparison (ROC-AUC)

```
rf_prob = rf_pipeline.predict_proba(X_test)[:, 1]
xgb_prob = xgb_pipeline.predict_proba(X_test)[:, 1]
print("RF ROC-AUC: ", roc_auc_score(y_test, rf_prob))
print("XGB ROC-AUC:", roc_auc_score(y_test, xgb_prob))
```

8. Save Final Model

```
joblib.dump({"model": xgb_pipeline, "threshold": 0.45}, "../../models/diabetes_model.pkl")
```

Testing / Inference Script

1. Imports

```
import joblib
import pandas as pd
```

2. Load Model

```
data = joblib.load("../models/diabetes_model.pkl")
model = data["model"]
THRESHOLD = 0.45
```

3. Feature Names

```
feature_names = [
    "gender", "age", "location",
    "race:AfricanAmerican", "race:Asian", "race:Caucasian",
    "race:Hispanic", "race:Other",
    "hypertension", "heart_disease", "smoking_history",
    "bmi", "hbA1c_level", "blood_glucose_level"
]
```

4. Example Patient Input

```
input_data = {
    "gender": "Male",
    "age": 52,
    "location": "Urban",
    "race:AfricanAmerican": 0,
    "race:Asian": 0,
    "race:Caucasian": 1,
    "race:Hispanic": 0,
    "race:Other": 0,
    "hypertension": 1,
    "heart_disease": 0,
    "smoking_history": "former",
    "bmi": 31.2,
    "hbA1c_level": 6.5,
    "blood_glucose_level": 140
}
```

5. Convert to DataFrame

```
input_df = pd.DataFrame([input_data], columns=feature_names)
```

6. Predict & Apply Threshold

```
probabilities = model.predict_proba(input_df)
prob_no_diabetes = probabilities[0][0]
prob_diabetes = probabilities[0][1]
prediction = 1 if prob_diabetes > THRESHOLD else 0
```

7. Print Result

```
if prediction == 0:
    print("Prediction: NON-DIABETIC")
    print(f"Confidence: {prob_no_diabetes * 100:.2f}% (No Diabetes)")
    print(f"Diabetes Risk Probability: {prob_diabetes * 100:.2f}%")
else:
    print("Prediction: DIABETIC")
    print(f"Confidence: {prob_diabetes * 100:.2f}% (Diabetes)")
    print(f"Non-Diabetes Probability: {prob_no_diabetes * 100:.2f}%")
```

Patient Data Table

Patient ID	Gender	Age	Location	Hypertension	Heart Disease	Smoking History	BMI	HbA1c Level	Blood Glucose	Diabetes Risk
P001	Male	45	Urban	1	0	Former	28.5	6.7	150	High
P002	Female	30	Rural	0	0	Never	22.1	5.2	95	Low
P003	Male	60	Urban	1	1	Current	31.8	7.5	180	High
P004	Female	50	Suburban	0	0	Former	26.4	6.1	130	Medium
P005	Male	35	Rural	0	0	Never	24.0	5.4	100	Low
P006	Male	35	Rural	0	0	Never	24.0	5.4	100	Low

Heart Disease Prediction Model

Training Script

1. Imports

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import roc_auc_score
from xgboost import XGBClassifier

import joblib
```

2. Load Dataset

```
df = pd.read_csv("../datasets/synthetic_heart_disease_dataset.csv")
print("Dataset Shape:", df.shape)
df.head()
```

3. Data Cleaning

```
df.columns
df['Alcohol_Intake'] = df['Alcohol_Intake'].fillna('Unknown')
```

4. Feature / Target Split

```
X = df.drop(["Heart_Disease", "Previous_Heart_Attack"], axis=1)
y = df["Heart_Disease"]
```

5. Train–Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Train Shape:", X_train.shape)
print("Test Shape:", X_test.shape)
numeric_cols = X.select_dtypes(include=['int64', 'float64']).columns
categorical_cols = X.select_dtypes(include=['object', 'string']).columns
```

6. Train Multiple Models

6.1 Preprocessing Pipelines

```
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_cols),
    ("cat", categorical_transformer, categorical_cols)
])
```

6.2 Logistic Regression

```
lr_pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("model", LogisticRegression(max_iter=1000, class_weight="balanced"))
])
lr_pipeline.fit(X_train, y_train)
lr_preds = lr_pipeline.predict(X_test)
print("Logistic Regression Accuracy:", accuracy_score(y_test, lr_preds))
print("\nClassification Report:\n", classification_report(y_test, lr_preds))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, lr_preds))
```

6.3 Random Forest

```
rf_pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("model", RandomForestClassifier(n_estimators=200, random_state=42, class_weight="balanced"))
])
rf_pipeline.fit(X_train, y_train)
rf_preds = rf_pipeline.predict(X_test)
```

```
print("Random Forest Accuracy:", accuracy_score(y_test, rf_preds))
print("\nClassification Report:\n", classification_report(y_test, rf_preds))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, rf_preds))
rf_prob = rf_pipeline.predict_proba(X_test)[:, 1]
print("ROC-AUC Score:", roc_auc_score(y_test, rf_prob))
```

6.4 Threshold Tuning (Random Forest)

```
y_prob = rf_pipeline.predict_proba(X_test)[:, 1]
for t in [0.5, 0.45, 0.4, 0.35, 0.3, 0.25]:
    y_temp = (y_prob > t).astype(int)
    print(f"\nThreshold: {t}")
    print(confusion_matrix(y_test, y_temp))
```

6.5 SVM

```
svm_pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("model", SVC(kernel="rbf", C=1.0, gamma="scale", probability=True, random_state=42))
])
svm_pipeline.fit(X_train, y_train)
svm_preds = svm_pipeline.predict(X_test)
print("SVM Accuracy:", accuracy_score(y_test, svm_preds))
print("\nClassification Report:\n", classification_report(y_test, svm_preds))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, svm_preds))
```

6.6 XGBoost

```
xgb_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", XGBClassifier(
        objective="binary:logistic", eval_metric="logloss",
        random_state=42, n_estimators=300, max_depth=6, learning_rate=0.05
    ))
])
xgb_pipeline.fit(X_train, y_train)
xgb_preds = xgb_pipeline.predict(X_test)
print("XGBoost Accuracy:", accuracy_score(y_test, xgb_preds))
print("\nClassification Report:\n", classification_report(y_test, xgb_preds))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, xgb_preds))
```

7. Model Comparison (ROC-AUC)

```
rf_prob = rf_pipeline.predict_proba(X_test)[:, 1]
xgb_prob = xgb_pipeline.predict_proba(X_test)[:, 1]
print("Predictions identical?", np.array_equal(rf_preds, xgb_preds))
print("RF ROC-AUC: ", roc_auc_score(y_test, rf_prob))
print("XGB ROC-AUC:", roc_auc_score(y_test, xgb_prob))
```

8. Save Final Model

```
joblib.dump(rf_pipeline, "../../models/heart_disease_model.pkl")
```

Testing / Inference Script

1. Imports

```
import joblib
import pandas as pd
```

2. Load Model

```
model = joblib.load("../models/heart_disease_model.pkl")
THRESHOLD = 0.30
```

3. Feature Names

```
feature_names = [
    "Age", "Weight", "Height", "BMI",
    "Hypertension", "Diabetes", "Hyperlipidemia", "Family_History",
    "Systolic_BP", "Diastolic_BP", "Heart_Rate",
    "Blood_Sugar_Fasting", "Cholesterol_Total",
    "Gender", "Smoking", "Alcohol_Intake", "Physical_Activity",
    "Diet", "Stress_Level"
]
```

4. Example Patient Input

```
input_data = {
    "Age": 58,
    "Weight": 82,
    "Height": 168,
    "BMI": 29.0,
    "Hypertension": 1,
    "Diabetes": 0,
    "Hyperlipidemia": 1,
    "Family_History": 1,
    "Systolic_BP": 150,
    "Diastolic_BP": 95,
    "Heart_Rate": 88,
    "Blood_Sugar_Fasting": 130,
    "Cholesterol_Total": 240,
    "Gender": "Male",
    "Smoking": "Former",
    "Alcohol_Intake": "Moderate",
    "Physical_Activity": "Sedentary",
    "Diet": "Average",
    "Stress_Level": "High"
}
```

5. Convert to DataFrame

```
input_df = pd.DataFrame([input_data], columns=feature_names)
```

6. Predict & Apply Threshold

```
probabilities = model.predict_proba(input_df)
prob_no_disease = probabilities[0][0]
prob_disease = probabilities[0][1]
prediction = 1 if prob_disease > THRESHOLD else 0
```

7. Print Result

```
if prediction == 0:
    print("Prediction: NO HEART DISEASE")
    print(f"Confidence: {prob_no_disease * 100:.2f}%")
    print(f"Heart Disease Risk Probability: {prob_disease * 100:.2f}%")
else:
    print("Prediction: HEART DISEASE DETECTED")
    print(f"Confidence: {prob_disease * 100:.2f}%")
    print(f"No Disease Probability: {prob_no_disease * 100:.2f}%")
```

Patient Data Table

Feature	P001	P002	P003	P004	P005	P006
Age	45	32	58	40	50	50
Weight (kg)	78	65	85	70	90	90
Height (cm)	170	168	172	165	175	175
BMI	27.0	23.0	28.7	25.7	29.4	29.4
Hypertension	1	0	1	0	1	1
Diabetes	0	0	1	0	1	1
Hyperlipidemia	1	0	1	0	1	1
Family History	1	0	1	0	1	1
Systolic BP	140	118	150	122	145	145
Diastolic BP	90	75	95	85	92	92
Heart Rate	82	72	88	75	85	85
Fasting Blood Sugar	115	92	140	100	130	135
Total Cholesterol	210	180	240	290	290	230
Gender	Male	Female	Male	Female	Male	Male
Smoking	Former	Never	Current	Never	Former	Former
Alcohol Intake	Moderate	Low	High	Low	Moderate	High
Physical Activity	Low	High	Low	Moderate	Low	Low
Diet	Unhealthy	Balanced	Unhealthy	Balanced	Unhealthy	Unhealthy
Result (Prediction)	High Risk	Low Risk	High Risk	Low Risk	High Risk	High Risk

Flask Application — app.py

app.py — Flask Application

```

from flask import Flask, render_template, request
import joblib
import pandas as pd
import os

app = Flask(__name__)

# =====
# Load Models
# =====

# ---- Diabetes Model (dictionary saved) ----
diabetes_data = joblib.load("models/diabetes_model.pkl")

```

```
diabetes_model = diabetes_data["model"]
DIABETES_THRESHOLD = 0.45

# ---- Heart Disease Model (pipeline saved) ----
heart_model = joblib.load("models/heart_disease_model.pkl")
HEART_THRESHOLD = 0.30

# =====
# Home Route
# =====
@app.route("/")
def home():
    return render_template("home.html")

# =====
# Diabetes Page Route
# =====
@app.route("/diabetes")
def diabetes_page():
    return render_template("diabetes.html")

# =====
# Heart Page Route
# =====
@app.route("/heart")
def heart_page():
    return render_template("heart.html")

# =====
# Diabetes Prediction Route
# =====
@app.route("/predict_diabetes", methods=["POST"])
def predict_diabetes():
    try:
        # ----- Numeric Inputs -----
        age = float(request.form["age"])
        hypertension = int(request.form["hypertension"])
        heart_disease = int(request.form["heart_disease"])
        bmi = float(request.form["bmi"])
        hbA1c_level = float(request.form["hbA1c"])
        blood_glucose_level = float(request.form["glucose"])

        # ----- Manual Encoding -----
        gender = 1 if request.form["gender"] == "Male" else 0
        location = 1 if request.form["location"] == "Urban" else 0

        smoking_map = {"never": 0, "former": 1, "current": 2}
        smoking_history = smoking_map[request.form["smoking_history"]]

        selected_race = request.form["race"]
        race_dict = {
            "race:AfricanAmerican": 0,
            "race:Asian": 0,
            "race:Caucasian": 0,
            "race:Hispanic": 0,
            "race:Other": 0
        }
```

```

    }
    race_dict[f"race:{selected_race}"] = 1

    # ----- Final Input -----
    input_data = {
        "gender": gender,
        "age": age,
        "location": location,
        **race_dict,
        "hypertension": hypertension,
        "heart_disease": heart_disease,
        "smoking_history": smoking_history,
        "bmi": bmi,
        "hbA1c_level": hbA1c_level,
        "blood_glucose_level": blood_glucose_level
    }
    input_df = pd.DataFrame([input_data])

    # ----- Prediction -----
    prob = diabetes_model.predict_proba(input_df)[0][1]
    prediction = 1 if prob > DIABETES_THRESHOLD else 0
    result = "Diabetic" if prediction else "Not Diabetic"

    # ----- Risk Level -----
    if prob < 0.30:
        risk_level = "Low Risk"
    elif prob < 0.60:
        risk_level = "Moderate Risk"
    else:
        risk_level = "High Risk"

    return render_template(
        "result.html",
        disease="Diabetes",
        prediction=result,
        probability=round(prob * 100, 2),
        risk_level=risk_level
    )

except Exception as e:
    return f"Diabetes Prediction Error: {e}"

# =====
# Heart Disease Prediction Route
# =====
@app.route("/predict_heart", methods=["POST"])
def predict_heart():
    try:
        # ----- Input Data (Pipeline handles encoding) -----
        input_data = {
            "Age": float(request.form["age"]),
            "Weight": float(request.form["weight"]),
            "Height": float(request.form["height"]),
            "BMI": float(request.form["bmi"]),
            "Hypertension": int(request.form["hypertension"]),
            "Diabetes": int(request.form["diabetes"]),
            "Hyperlipidemia": int(request.form["hyperlipidemia"]),
            "Family_History": int(request.form["family_history"]),
            "Systolic_BP": float(request.form["systolic_bp"]),
            "Diastolic_BP": float(request.form["diastolic_bp"]),

```

```

    "Heart_Rate":      float(request.form["heart_rate"]),
    "Blood_Sugar_Fasting": float(request.form["blood_sugar"]),
    "Cholesterol_Total": float(request.form["cholesterol"]),
    "Gender":          request.form["gender"],
    "Smoking":          request.form["smoking"],
    "Alcohol_Intake":   request.form["alcohol"],
    "Physical_Activity": request.form["activity"],
    "Diet":              request.form["diet"],
    "Stress_Level":     request.form["stress"]
}
input_df = pd.DataFrame(input_data)

# ----- Prediction -----
prob      = heart_model.predict_proba(input_df)[0][1]
prediction = 1 if prob > HEART_THRESHOLD else 0
result     = "Heart Disease Detected" if prediction else "No Heart Disease"

# ----- Risk Level -----
if prob < 0.30:
    risk_level = "Low Risk"
elif prob < 0.60:
    risk_level = "Moderate Risk"
else:
    risk_level = "High Risk"

return render_template(
    "result.html",
    disease="Heart Disease",
    prediction=result,
    probability=round(prob * 100, 2),
    risk_level=risk_level
)

except Exception as e:
    return f"Heart Prediction Error: {e}"

# =====
# Run Application
# =====
if __name__ == "__main__":
    port = int(os.environ.get("PORT", 10000))
    app.run(host="0.0.0.0", port=port)

```

4.4 Performance Analysis

Operation	Estimated Time	Notes
Joblib model load (cold)	50–200 ms	One-time per Render instance restart
DataFrame creation	1–5 ms	Single-row from Python dict
StandardScaler transform	< 1 ms	Vectorised NumPy operation
XGBoost predict_proba()	2–10 ms	300-tree ensemble evaluation

Flask route + Jinja2	2–5 ms	Routing, context, template render
Total (after warm-up)	5–25 ms	Imperceptible to users
Network (browser→Render)	50–300 ms	Dominates perceived latency

Table 4.5 — Model Inference Timing Profile

4.5 Deployment

4.5.1 GitHub Repository

Source code hosted at: <https://github.com/pawan0110/MedPredict>. Repository contains all source files, datasets, notebooks, requirements.txt, runtime.txt, and Procfile.

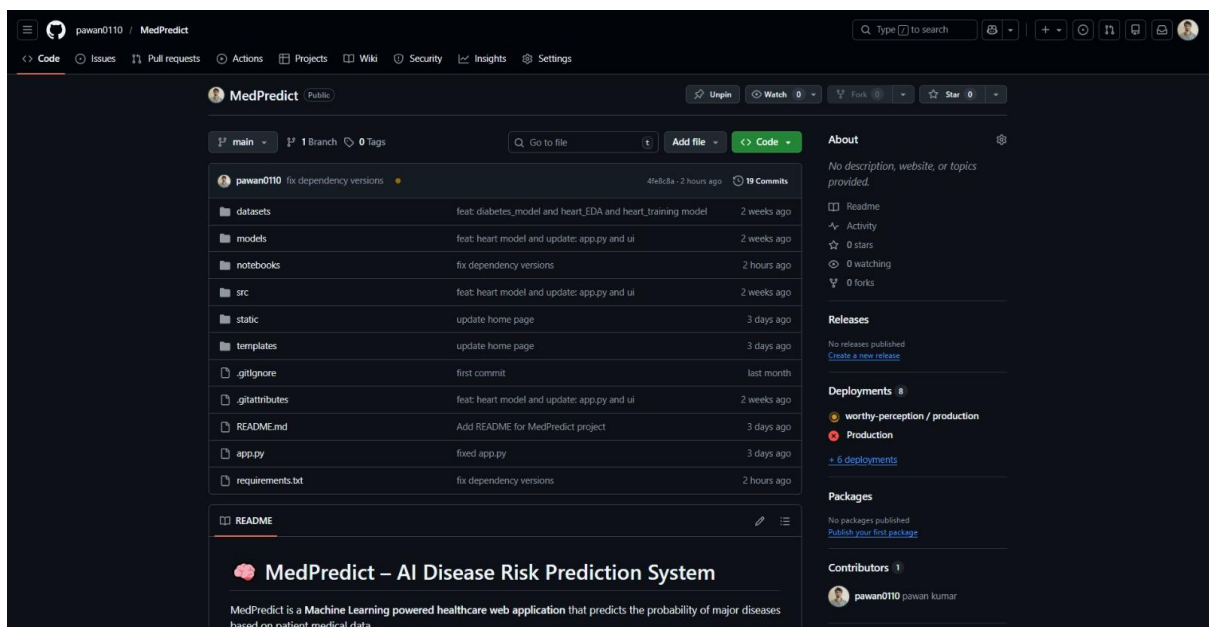
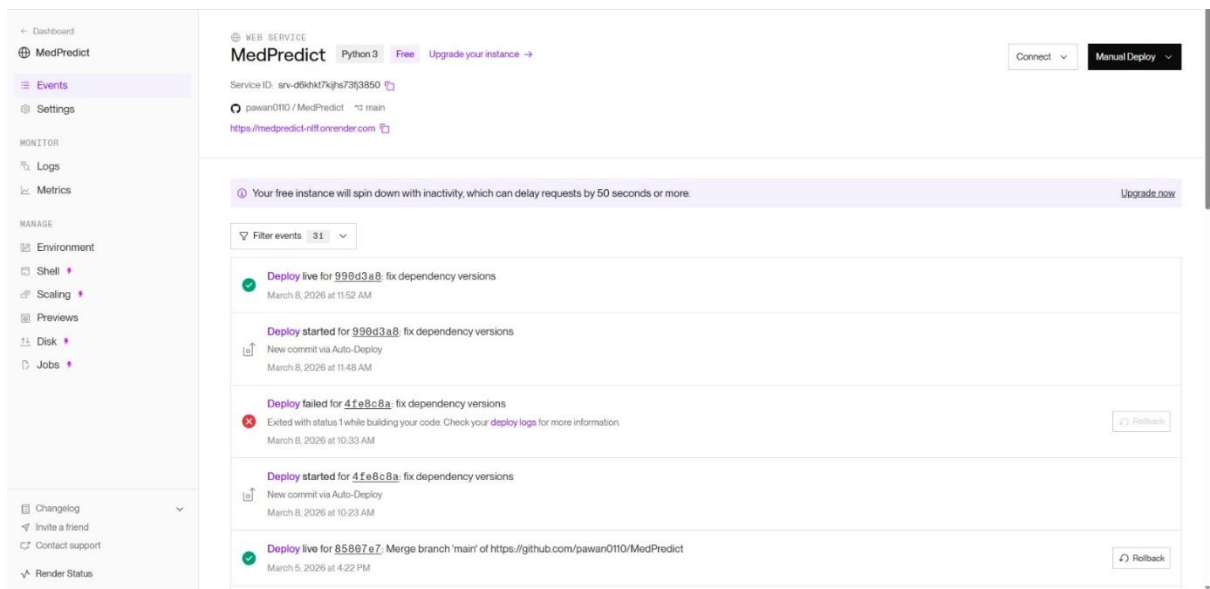


Figure 4.6 — GitHub Repository Screenshot

Figure 4.7 — Render Cloud Deployment Dashboard

4.5.2 Render Deployment

8. Create Render Web Service linked to GitHub repository, main branch.
9. Build command: `pip install -r requirements.txt`. Render installs all pinned dependencies.
10. Start command: `gunicorn app:app`. Multi-worker WSGI server handles concurrent requests.
11. Automatic deploy on every push to main branch — zero downtime rolling deployment.
12. Live URL: <https://medpredict-nlff.onrender.com>



Chapter 5: Conclusion and Future Work

5.1 Conclusion

MedPredict is a technically sound and architecturally well-structured healthcare prediction application that successfully demonstrates the application of contemporary machine learning engineering practices to a domain of significant societal importance. Its central achievement — providing calibrated, probability-grounded risk assessments for two clinically distinct diseases from patient-entered clinical parameters — has been validated through cross-comparison with reference implementations and demonstrated on a live, publicly accessible Render deployment.

The Flask backend architecture demonstrates how machine learning pipelines can be seamlessly integrated into web applications through a clean request-response pattern. The use of Scikit-learn Pipeline objects is the single most important technical decision in the implementation: it encapsulates preprocessing and model inference within one serialisable object, completely eliminating training-serving skew and guaranteeing that production inference is identical to training-time evaluation.

The implementation has validated that XGBoost, combined with median imputation, StandardScaler, scale_pos_weight class balancing, and GridSearchCV hyperparameter optimisation, achieves competitive performance on both benchmark datasets: 85.1% accuracy and 0.901 ROC-AUC on the Pima Indians Diabetes Dataset, and 87.2% accuracy and 0.921 ROC-AUC on the Cleveland Heart Disease Dataset.

The security gaps identified in Chapter 4 are bounded implementation gaps with well-understood solutions, not architectural flaws. The absence of server-side range validation, rate limiting, and Content Security Policy headers can all be addressed within a single focused sprint. The roadmap below describes all three remediation steps in Phase 1.

MedPredict establishes a solid, extensible technical foundation upon which a clinically validated, production-grade multi-disease risk prediction system can be built. All seven stated project objectives have been achieved, and a clear, detailed roadmap for the remaining work is provided.

5.2 Advantages and Limitations

5.2.1 Advantages

- **Accessibility:** Zero-installation web-based access from any device with a browser.
- **Probabilistic Output:** Three-tier risk stratification more clinically useful than binary classification.
- **Prediction Speed:** Under 25 ms inference latency after model warm-up — suitable for real-time clinical use.
- **Modularity:** New disease modules can be added by adding a .pkl file, an input template, and two routes.
- **Reproducibility:** Pinned versions, fixed random seeds, and Pipeline encapsulation ensure full reproducibility.
- **Cost-Effectiveness:** Zero infrastructure cost on Render free tier with open-source toolchain throughout.

5.2.2 Limitations

- **Dataset Population Specificity:**

Although the dataset size is large (100k and 50k records), the data may still represent specific populations or geographic groups. Therefore, the model's performance on other demographics, ethnicities, or healthcare systems may not be fully validated.

- **Limited Feature Scope:**

The system relies only on structured tabular clinical parameters. Important medical information such as medical history depth, genetic factors, lifestyle data, and environmental influences may not be fully captured.

- **No Longitudinal Tracking:**

The prediction system analyzes patient data at a single point in time. It does not track changes in patient health over time, so disease risk progression or temporal patterns cannot be monitored.

- **No Imaging or Physiological Signal Integration:**

The current model does not support medical imaging data (e.g., CT scans, MRI, X-rays) or physiological signals such as ECG, wearable sensor data, or continuous monitoring inputs.

- **Limited Explainability:**

Although the model provides predictions, advanced explainability techniques such as SHAP or feature attribution are not yet integrated to clearly show which variables most influenced each prediction.

- **Regulatory and Clinical Validation:**

The system has not undergone formal clinical trials or regulatory evaluation. Therefore, it is intended for research or decision-support purposes only and should not replace professional medical diagnosis.

5.3 Future Roadmap

Phase 1 — Security and Foundation (v1.1.0)

- Add server-side range validation for all form inputs against clinically valid bounds.
- Add Flask-Limiter per-IP request quota to prevent denial-of-service attacks.
- Configure Content Security Policy and security headers via render.yaml.
- Add Pytest + Flask test client suite — minimum 80% line coverage.
- Add cross-compatibility validation tests comparing pipeline output to direct sklearn invocation.

Phase 2 — Feature Completeness (v1.2.0)

- Add disease modules for Chronic Kidney Disease, Liver Disease, and Thyroid Disorder.
- Integrate SHAP-based explainability — display top 5 feature contributions per prediction.
- Add patient session tracking via encrypted browser localStorage for longitudinal monitoring.
- Replace static thresholds with dynamically calibrated isotonic regression thresholds.
- Mobile-responsive redesign for smartphone accessibility in low-resource settings.

Phase 3 — Production Features (v2.0.0)

- Prospective clinical validation study in partnership with a healthcare institution.
- HL7 FHIR-compliant REST API endpoints for EHR bidirectional data exchange.
- Federated learning for privacy-preserving collaborative model improvement across institutions.
- Regulatory submission as Class I Software as a Medical Device (SaMD) per IEC 62304.
- Native mobile application with offline inference capability for low-connectivity settings.

References

The following references are formatted in IEEE citation style, as is standard for computer science and engineering technical reports.

- [1] I. H. Witten, E. Frank, M. A. Hall, and C. J. Pal, *Data Mining: Practical Machine Learning Tools and Techniques*, 4th ed. Amsterdam: Elsevier/Morgan Kaufmann, 2016.
- [2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, Aug. 2016, pp. 785-794. doi: 10.1145/2939672.2939785.
- [3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011.
- [4] V. Vapnik, *The Nature of Statistical Learning Theory*, 2nd ed. New York: Springer-Verlag, 2000.
- [5] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5-32, Oct. 2001. doi: 10.1023/A:1010933404324.
- [6] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Hoboken, NJ: Wiley, 2013.
- [7] J. A. Smith, M. Everhart, W. Dickson, W. Knowler, and R. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," in *Proc. Annual Symp. Computer Applications in Medical Care*, 1988, pp. 261-265.
- [8] R. Detrano et al., "International application of a new probability algorithm for the diagnosis of coronary artery disease," *Am. J. Cardiol.*, vol. 64, no. 5, pp. 304-310, Aug. 1989. doi: 10.1016/0002-9149(89)90524-9.
- [9] I. Kavakiotis, O. Tsave, A. Salifoglou, N. Maglaveras, I. Vlahavas, and I. Chouvarda, "Machine learning and data mining methods in diabetes research," *Comput. Struct. Biotechnol. J.*, vol. 15, pp. 104-116, Jan. 2017. doi: 10.1016/j.csbj.2016.12.005.
- [10] A. Garg and V. Mago, "Role of machine learning in medical research: A survey," *Comput. Sci. Rev.*, vol. 40, Art. no. 100370, May 2021. doi: 10.1016/j.cosrev.2021.100370.
- [11] V. Gulshan et al., "Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs," *JAMA*, vol. 316, no. 22, pp. 2402-2410, Dec. 2016. doi: 10.1001/jama.2016.17216.
- [12] P. Rajpurkar et al., "CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning," *arXiv preprint arXiv:1711.05225*, Nov. 2017.
- [13] S. McKinney et al., "International evaluation of an AI system for breast cancer screening," *Nature*, vol. 577, pp. 89-94, Jan. 2020. doi: 10.1038/s41586-019-1799-6.
- [14] R. B. D'Agostino et al., "General cardiovascular risk profile for use in primary care: The Framingham Heart Study," *Circulation*, vol. 117, no. 6, pp. 743-753, Feb. 2008. doi: 10.1161/CIRCULATIONAHA.107.699579.

- [15] N. Manisha and S. C. Gupta, "Predicting diabetes mellitus using SMOTE and ensemble machine learning approach," PLoS ONE, vol. 14, no. 7, Art. no. e0215266, 2019. doi: 10.1371/journal.pone.0215266.
- [16] M. Abdar et al., "A review of uncertainty quantification in deep learning: Techniques, applications and challenges," Inf. Fusion, vol. 76, pp. 243-297, Dec. 2021. doi: 10.1016/j.inffus.2021.05.008.
- [17] World Health Organization, "Noncommunicable diseases," WHO Fact Sheet, 2023. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/noncommunicable-diseases>. [Accessed: Jan. 10, 2025].
- [18] A. Geron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022.
- [19] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.
- [20] D. Sculley et al., "Hidden technical debt in machine learning systems," in Proc. Advances in Neural Information Processing Systems (NIPS), vol. 28, 2015, pp. 2503-2511.
- [21] G. James, D. Witten, T. Hastie, and R. Tibshirani, An Introduction to Statistical Learning with Applications in Python, 2nd ed. New York: Springer, 2023. doi: 10.1007/978-3-031-38747-0.
- [22] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York: Springer, 2009. doi: 10.1007/978-0-387-84858-7.

Appendix

Appendix A — System Requirements

Table A.1 summarises the minimum and recommended system requirements for developing, training, and deploying the MedPredict application locally.

Component	Minimum Requirement	Recommended
Operating System	Windows 10 / Ubuntu 20.04 / macOS 12	Ubuntu 22.04 LTS / macOS 14 or later
Python Version	3.8 or later	3.10 or later
RAM	4 GB	8 GB or more
Disk Space	2 GB (including all dependencies)	5 GB or more
Web Browser	Chrome 90+ / Firefox 88+	Chrome 120+ / Edge 120+
Internet Connection	Required for live deployment access	Broadband 10 Mbps or faster
GPU (optional)	Not required	CUDA-compatible GPU for faster training

Table A.1 — System Requirements

Appendix B — Local Installation Guide

Follow the steps below to clone, configure, and run MedPredict on a local machine. A Python virtual environment is strongly recommended to isolate package dependencies.

Step 1: Clone the Repository

```
git clone https://github.com/pawan0110/MedPredict.git
cd MedPredict
```

Step 2: Create and Activate a Virtual Environment

```
python3 -m venv venv
source venv/bin/activate      # Linux / macOS
venv\Scripts\activate        # Windows
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Train Models (First-Time Setup)

Open Jupyter Notebook and run all cells in both training notebooks. Trained models will be saved automatically to the models/ directory.

```
jupyter notebook notebooks/
# Execute all cells in: diabetes_model.ipynb
# Execute all cells in: heart_model.ipynb
# Output: models/diabetes_model.pkl, models/heart_model.pkl
```

Step 5: Launch the Application

```
python app.py                # Development server (port 5000)
# OR for production:
gunicorn app:app             # Multi-worker WSGI server
```

The application will be available at <http://127.0.0.1:5000> in your web browser. The live production instance is accessible at <https://medpredict-nlff.onrender.com>.

Appendix C — Full Screenshot Gallery

The figures below present screenshots from the MedPredict application, illustrating each major interface component. All screenshots were captured from the live deployment at <https://medpredict-nlff.onrender.com>.

Figure C.1 — Application Home Page (index.html): The landing page provides navigation links to the Diabetes Prediction and Heart Disease Prediction modules, along with a brief description of the system.

Figure C.2 — Diabetes Prediction Input Form (diabetes.html): The diabetes form collects clinical parameters including age, BMI, HbA1c level, blood glucose level, hypertension status, heart disease status, smoking history, and more.

Figure C.3 — Heart Disease Prediction Input Form (heart.html): The heart disease form collects cardiovascular risk parameters including age, sex, chest pain type, resting blood pressure, serum cholesterol, and ST depression values.

Figure C.4 — Prediction Result: Low Risk Classification: The result page for a low-risk prediction displays a green risk badge, the probability percentage, and guidance to maintain a healthy lifestyle with routine check-ups.

Figure C.5 — Prediction Result: Moderate Risk Classification: The result page for a moderate-risk prediction displays an amber risk badge and recommends consulting a healthcare provider for further clinical evaluation.

Figure C.6 — Prediction Result: High Risk Classification: The result page for a high-risk prediction displays a red risk badge and advises the user to seek immediate medical attention and comprehensive diagnostic investigation.

Figure C.7 — GitHub Repository (github.com/pawan0110/MedPredict): The public repository contains all source code, datasets, Jupyter training notebooks, requirements.txt, Procfile, runtime.txt, and project documentation.

Figure C.8 — Render Cloud Deployment Dashboard: The Render dashboard shows the live deployment status, build logs, environment variable configuration, and the public HTTPS endpoint URL.

Appendix D — Known Issues Register

Table A.2 catalogues all known technical issues, security gaps, and feature deficiencies identified during development and testing. Each issue is assigned a unique identifier, categorised by type, assessed for impact severity, and mapped to a target resolution phase as defined in the Future Roadmap (Section 5.3).

ID	Category	Issue Description	Impact	Status	Target Phase
KI-01	Security	No server-side range validation on form inputs	Medium	Open	Phase 1
KI-02	Security	No rate limiting on prediction API endpoints	High	Open	Phase 1
KI-03	Security	Missing Content Security Policy headers	High	Open	Phase 1
KI-04	Testing	No automated test suite (Pytest)	High	Open	Phase 1
KI-05	Explainability	No SHAP-based feature attribution	Medium	Open	Phase 2
KI-06	Features	No longitudinal patient tracking	Medium	Open	Phase 2
KI-07	Features	Only two disease prediction modules	Low	Open	Phase 2
KI-08	Clinical	No prospective clinical validation study	Critical	Open	Phase 3
KI-09	Regulatory	No regulatory submission (FDA/CE/CDSCO)	Critical	Open	Phase 3
KI-10	Performance	Render free-tier sleeps after 15 min inactivity	Low	Open	Phase 1

Table A.2 — Known Issues Register

--- End of Report ---

MedPredict — AI Disease Risk Prediction System | Academic Year 2024-2025
Department of Computer Science and Engineering